

TD6 – éléments de corrigé (exercice 2)

Structure : **Etudiant**

```
{
    champ _nom : chaîne de caractères
    champ _prenom : chaîne de caractères
    champ _groupe : entier
    champ _nbNotes : entier
    champ _notes : tableau 1D de 6 entiers
    champ _moyenne : réel

    Constructeur : Etudiant
    Entrée :
        - nom : chaîne de caractères
        - prenom : chaîne de caractères
        - groupe : entier

    Début
        |
        |     _nom = nom
        |     _prenom = prenom
        |     _groupe = groupe
        |     _nbNotes = 0
        |     _notes = tableau 1D de 6 entiers initialisé avec des 0
        |     _moyenne = 0.0
    Fin
}
```

Procédure : **AjouterNote**

```
Entrée :
    - référence etu : structure Etudiant
    - note : réel
    - i : entier

Sortie :
    void

Début
    |
    |     Si etu._nbNotes < Taille(etu._notes) Alors
    |         |
    |         |     etu._notes[etu._nbNotes] = note
    |         |     etu._nbNotes = etu._nbNotes + 1
    |         |     etu._moyenne = 0.0
    |         |     Pour i de 0 à etu._nbNotes Faire
    |         |         |     etu._moyenne = etu._moyenne + etu._notes[i]
    |         |     Fin Pour
    |         |     etu._moyenne = etu._moyenne / etu._nbNotes
    |     Fin Si
Fin
```

Fonction : LireEtudiants

Entrée : - chemin : chaîne de caractères

Variables : - etudiants : tableau de structure Etudiant

- numEtu, i : entiers

- ligne : chaîne de caractères

- lignes, elements : tableaux 1D de chaînes de caractères

Sortie : tableau 1D de structure Etudiant

Début

OuvrirFichier f depuis chemin en Lecture

lignes = f.LireToutesLesLignes()

etudiants = tableau 1D de Taille(lignes) structure Etudiant

numEtu = 0

Pour Chaque chaîne de caractères ligne dans lignes Faire

elements = ligne.Séparer(",")

etu = Etudiant(elements[0], elements[1], Entier(elements[2]))

Pour i de 0 à 5 Faire

AjouterNote(référence etu, Réel(elements[i + 3]))

Fin Pour

etudiants[numEtu] = etu

numEtu = numEtu + 1

Fin Pour Chaque

FermerFichier f

Retourner etudiants

Fin

Procédure : CalculerMoyennePromotion

Entrée : - etudiants : tableau 1D de structure Etudiant

Variables : - moyenne : réel

Sortie : réel

Début

moyenne = 0.0

Pour Chaque structure Etudiant etu dans etudiants Faire

moyenne = moyenne + etu._moyenne

Fin Pour Chaque

Retourner moyenne / Taille(etudiants)

Fin

Procédure : AfficherMoyennesGroupes

Entrée : - etudiants : tableau 1D de structure Etudiant

Variables : - groupe, numEtu : entiers

- moyenne : réel

Sortie : void

Début

Si Taille(etudiants) < 1 Alors

Retourner

Fin Si

groupe = etudiants[0]._groupe;

numEtu = 0

moyenne = 0.0

Pour Chaque structure Etudiant etu dans etudiants Faire

Si etu._groupe != groupe Alors

moyenne = moyenne / numEtu

Afficher("La moyenne du groupe " + groupe + " est " + moyenne)

moyenne = etu._moyenne

numEtu = 1

groupe = etu._groupe

Sinon

moyenne = moyenne + etu._moyenne

numEtu = numEtu + 1

Fin Si

Fin Pour Chaque

moyenne = moyenne / numEtu

Afficher("La moyenne du groupe " + groupe + " est " + moyenne)

Fin

Algorithme 1 : AfficherMajorChaqueUE

Entrée : - étudiants : tableau 1D de structure Etudiant

Variables : - ue : entier
- major : structure Etudiant

Sortie : void

Début

```
Pour ue de 0 à 5 Faire
    major = étudiants[0]
    Pour Chaque structure Etudiant etu dans étudiants Faire
        Si etu._notes[ue] > major._notes[ue] Alors
            major = etu
        Fin Si
    Fin Pour
    Afficher("Major de l'UE " + ue + " : " + major._prenom + " " + major._nom +
        " avec une note de " + major._notes[ue] + "/20")
Fin Pour
```

Fin

Procédure : AjouterEtudiantPromotion

Entrée : - etu : structure Etudiant
- chemin : chaîne de caractères

Variables : - i : entier

Sortie : void

Début

```
Si Existe(chemin) Alors
    OuvrirFichier f depuis chemin en Ajout
        ligne = etu._nom + ";" + etu._prenom + ";" + etu._groupe + ";"
        Pour i de 0 à 4 Faire
            ligne = ligne + etu._notes[i] + ";"
        Fin Pour
        ligne = ligne + etu._notes[Taille(etu._notes) - 1]
        f.Ecrire(ligne)
    FermerFichier f
Fin Si
```

Fin

Algorithme : Main

Variables : - chemin : chaîne de caractères
- i : entier
- albert : structure Etudiant

Début

```
chemin = "Promotion.csv"
Si Existe(chemin) Alors
    étudiants = LireEtudiants(chemin)
    Pour i de 0 à Taille(etudiants) - 1 Faire
        Afficher(etudiants[i])
    Fin Pour
    Afficher("Moyenne de la promotion : " + CalculerMoyennePromotion(etudiants))
    AfficherMoyennesGroupes(etudiants)
    AfficherMajorChaqueUE(etudiants)
    albert = Etudiant("Einstein", "Albert", 3)
    AjouterNote(référence albert, 10.0)
    AjouterNote(référence albert, 15.0)
    AjouterNote(référence albert, 7.5)
    AjouterNote(référence albert, 7.5)
    AjouterNote(référence albert, 13.25)
    AjouterNote(référence albert, 18.0)
    AjouterEtudiantPromotion(albert, chemin)
Fin Si
```

Fin